

Software Development Management

How engineering teams ship software — and how to run the same playbook as a team of one, with AI agents.

Agile · Scrum · Tooling · Agentic Delivery



OPENING

Two things that shouldn't both be true.

- **Story 1 — Ninja Van.** We spent years deliberately splitting our code into ~100 separate repositories — and it was the right call. This year we spent months gluing some of it back together — *also* the right call.
- **Story 2 — the METR trial (2025).** Experienced developers using AI coding tools were *measured* to be **19% slower** — while *believing* they were **20% faster**.
- By the end of these two hours you'll know exactly why both happened — and what they mean for how *you* will ship.

| *Building software isn't the hard part. Coordinating the building of it is — and AI just rewrote the rules.*

WELCOME

You're here for data & AI — why care how dev teams run?

- **You'll work with engineering teams** — as an analyst, a data scientist, a PM, a founder. You'll need to know the difference between real delivery and delivery theatre.
- **You'll ship your own projects** — capstones, portfolio pieces, side products. The same disciplines that make teams effective are what keep a solo project alive beyond week two.
- **AI made code cheap.** The scarce skills now: deciding what to build, breaking it down, and judging what comes back.

| *Each part: how a 200-person org does it → the one-person version you can use tomorrow.*

Two hours · three exercises · one video · one live demo · one index card.

01	Opening · Welcome — delivering value, not just writing code	0:00	02	Part 1 · Why you can't just add people — the n^2 problem · Conway's Law	0:06
03	Part 2 · Agile · Scrum — the loop that works · 🎬 video · 🖋️ spot-the-theatre	0:16	04	Part 3 · Measuring teams — DORA · Goodhart · 🖋️ break-the-metric	0:40
05	☕ Stretch break (5 min)	1:00	06	Part 4 · The agentic shift — 🖥️ live demo · what AI changed at Ninja Van	1:05
07	Part 5 · Agile for one — your toolkit · 🖋️ plan-your-project	1:32	08	Recap · Q&A	1:52

● CHAT POLL · TAKE 60 SECONDS

What's the biggest team you've ever worked on — and did adding people make it faster?

| *What changed as the team grew?*

BLOCK 1

PART 1 OF 5 · WHY COORDINATION BEATS CODE

You can't just add people.



Communication paths grow as $n(n-1)/2$.

PEOPLE ON THE TEAM	COMMUNICATION PATHS
3	3
7	21
15	105
50	1,225

7 → 50 people multiplies communication ~58x, not ~7x. Most of 'software management' exists to fight this number.

“Adding manpower to a late software project makes it later.”

— FRED BROOKS · THE MYTHICAL MAN-MONTH (1975)

The fix: small teams that own something end-to-end.

- **The two-pizza rule** (Amazon): if two pizzas can't feed the team, it's too big — about 5–9 people.
- **A team owns something** — a service, a product area, a customer journey — behind a clear boundary. Ownership + boundary makes a team, not a shared job title.
- **Organisations scale by adding teams, not people** — small units that talk through clean interfaces.

| *Same trick as software architecture — modules with clean interfaces — applied to people.*

Your software ends up shaped like your org chart.

- "Organisations design systems that mirror their own communication structure." — Melvin Conway, 1967.
- Separate payments and checkout teams → two services with an API between them. Two teams building a compiler → a two-pass compiler.
- **The Inverse Conway Manoeuvre:** deliberately design the *team* structure to get the *architecture* you want.

| Park this law — it's the key to opening Story 1, and it returns in Part 4 when the 'team' we restructure for is an AI agent.

BLOCK 2

PART 2 OF 5 · AGILE & SCRUM

Ship small. Get feedback. Adjust.

Waterfall vs Agile — the cost of being wrong.

WATERFALL — PLAN IT ALL, THEN BUILD

Requirements → Design → Build → Test → Ship · One big pass, often months long · You find out it's wrong **at the end** — when it's most expensive

AGILE — SMALL LOOPS

Build a slice → ship → get feedback → adjust → repeat · Loops of days or weeks, not months · **Wrong is cheap** — it's caught small and early

Four values — the spirit, not the ritual.

- Individuals & interactions *over* processes & tools
- Working software *over* comprehensive documentation
- Customer collaboration *over* contract negotiation
- Responding to change *over* following a plan

| A rebellion against big-upfront-plan waterfall. ⚠ Much of what's sold as 'Agile™' today is exactly what it rebelled against.

Scrum vs Kanban.

SCRUM — RHYTHM & COMMITMENT

Fixed-length **sprints** (1–2 weeks) · Roles: **Product Owner** (what) · **Scrum Master** (process) · Development Team (how) · Ceremonies: planning · stand-up · review · **retro**

KANBAN — FLOW & FLEXIBILITY

No sprints — **continuous flow** · **WIP limits** — finish before you start · **Pull** work when you have capacity · *Exploratory work — analysis, experiments — usually fits flow better than sprint commitments.*

Anatomy of a two-week sprint.

01 Day 1 — Sprint planning

Pull work from the backlog; commit to a realistic slice.

02 Every day — Stand-up

10 minutes. What I did · what I'm doing · what's blocking me.

03 Mid-sprint — Refinement

Groom and estimate what's coming next.

04 Last day — Review + Retro

Demo what shipped to stakeholders; then improve how the team works.

| *The rhythm is the point — the same heartbeat every two weeks. And the only stand-up question that really matters: 'what's blocking me?'*

What vague requirements sound like.



"The Expert" — Lauris Beinerts (2014)

<https://www.youtube.com/watch?v=BKorP55Aqvg>

Start at 0:00 · play ~2.5 min (stop at ~2:45)

'Seven red lines, all perpendicular, some with green ink, some with transparent ink.' Every stakeholder meeting has an Expert — someone handed requirements that sound precise but mean nothing. The antidote is on the next slide: user stories with **acceptance criteria** that define *done* before anyone draws a line. And hold this thought for Part 4: an AI agent is the *ultimate* Expert's engineer — it won't push back, it will just confidently draw the seven red lines.

Vague requirements produce confident nonsense — from humans and from AI.

User stories & estimation — the antidote to the Expert.

USER STORIES

"As a **[user]**, I want **[goal]**, so that **[benefit]**." · **Acceptance criteria** define "done" up front — before any code · Small, vertical, shippable slices

ESTIMATION (THE SHORT VERSION)

Story points — relative size, not hours (humans are terrible at hours) · **Velocity** = points/sprint — a trend for forecasting, *never* a target · *Watch what happens in Part 3 when it becomes one...*

The retro is where teams fix themselves.



Went well

Shipped the tracking API ahead of schedule · Pairing on the migration cut review time · Zero production incidents this sprint



Didn't go well

3 stories carried over — auth refactor underestimated · Flaky e2e tests blocked the pipeline twice · Stand-ups drifted to 25 minutes



Actions · owner

Cap WIP at 3 per dev — Mei · Quarantine + fix flaky tests by Wed — Arjun · Timebox stand-up to 10 min — Sarah

The rule: every retro produces 1–2 actions, each with an owner — otherwise it's just venting. And it must be blameless, or people go quiet.

Real agile — or theatre?

Four teams all say they're agile. With your neighbour: which are genuinely agile, which are theatre — and what's the tell?

- **Team A** — 25-min daily stand-up where everyone reports status to the manager. Sprints always '100% complete' — unfinished work quietly rolls over.
- **Team B** — no sprints at all. A kanban board with WIP limits, ships several times a week, monthly retro that produces actions.
- **Team C** — every ceremony by the book. Velocity target raised each sprint; estimates have doubled in six months; delivery hasn't sped up.
- **Team D** — two-week sprints, 10-min stand-ups, demos to real users at every review, retro actions with owners.

| 3 min discuss · 4 min debrief. The tell is always the same: is there a feedback loop that actually changes anything?

Where the work comes from: roadmaps & priorities.

01 Outcomes over outputs

'Reduce failed deliveries 10%' — not 'build feature X.' Outcomes leave room to learn.

02 Now / Next / Later

Honest buckets instead of fake-precise dates nine months out.

03 OKRs

A qualitative objective + measurable key results, reviewed quarterly.

04 Prioritise with RICE

Reach × Impact × Confidence ÷ Effort — kills loudest-voice-wins.

| *A roadmap is a communication tool, not a dated contract. It's allowed to change.*

RICE — scoring what to build next.

FEATURE	REACH	IMPACT	CONF.	EFFORT	SCORE
Faster checkout	8k	2	80%	3 wk	4,267
Fraud alerts	2k	3	70%	2 wk	2,100
Dark mode	5k	0.5	100%	2 wk	1,250

Score = (Reach × Impact × Confidence) ÷ Effort. It doesn't decide for you — it makes trade-offs visible. Use it on your capstone: ten ideas, two weekends.

BLOCK 3

PART 3 OF 5 · MEASURING PERFORMANCE

Measure the system — never
the person.

● CHAT POLL · TAKE 60 SECONDS

How would you measure whether a dev team is productive?

| *Shout it out — what would you count?*

Everything obvious measures activity, not outcome.

- **Lines of code** — the best change often *deletes* code. Rewards bloat.
- **Commits / hours** — reward busywork and presenteeism.
- **Tickets closed** — invites slicing work into trivial tickets.
- **Story points** — a planning aid, not a scoreboard. (Remember Team C.)
- **And AI broke these for good** — an agent emits 10,000 plausible lines before lunch.

| "Measuring programming progress by lines of code is like measuring aircraft progress by weight." — attrib. Bill Gates

Ninety seconds of jargon: CI/CD, PRs, tests.



*****CI/CD**** — build & test automatically on every change (CI); automate the path to production (CD). ****PR (pull request)**** — a proposed code change plus the review conversation around it; 'code review' happens here, and unreviewed code doesn't merge. ****Unit test**** — a small program that automatically checks another program: pass or fail, identically, every time. Together they shorten the time between making a change and learning whether it worked.*

DORA: the four metrics that survived the research.

METRIC	QUESTION IT ANSWERS	AXIS
Deployment Frequency	How often do we ship to production?	Speed
Lead Time for Changes	Commit → running in prod: how long?	Speed
Change Failure Rate	What % of deploys cause a problem?	Stability
Time to Restore (MTTR)	When we break, how fast do we recover?	Stability

From Google's DORA research program (the book Accelerate). They resist gaming because they're outcomes of the whole system — not of any one person.



The surprise finding: speed and stability go together.

- Elite teams deploy **on demand** with lead times **under a day** — *and* fail less *and* recover in under an hour.
- Small, frequent changes are easier to verify and easier to roll back — shipping daily is usually *safer*, not riskier.
- Watch lead time to find where work **waits** — it's almost never the coding. It's review queues and handoffs.

| *The slow part of software is waiting, not typing — and AI is about to make the waiting worse.*

*“When a measure becomes a target, it
ceases to be a good measure.”*

— GOODHART'S LAW — THE CARDINAL RULE OF ENGINEERING METRICS

● ✎ EXERCISE 2 · 8 MIN · WITH YOUR NEIGHBOUR

Design a metric — then break it.

Your pair runs a **6-person team** shipping a customer-facing app.

- **1 · Pick one metric** you'd track for the team's health — steal from DORA if you like.
- **2 · Play the lazy genius:** how would you hit the number *without* doing better work?
- **3 · Add one guardrail** — a paired metric or a rule — that makes the gaming pointless.

| 4 min design · 4 min share-back. DORA's own trick: pair every speed metric with a stability metric, so juicing one wrecks the other.

A flashlight on the system — never a scoreboard on people.

- The moment engineers believe the dashboard watches *them*, they hide problems, game numbers, and stop taking risks. We learned this deploying analytics at Ninja Van.
- The rules that saved it: **aggregate, never individual · owned by the team · read in retros, never in performance reviews.**
- **The AI era raised the stakes:** DORA (2024) found teams adopting AI got *individually* faster while delivery throughput and stability initially **dropped**.

| DORA 2025's headline: AI is an amplifier — it magnifies a healthy system, and magnifies a dysfunctional one.

INTERMISSION

5^{min}

Back soon. Resume at the time on screen.

Halftime — the story so far.

- Coordination, not code, is the hard part — so: small teams, clean boundaries (n^2 , Conway).
- Agile is 'ship small, get feedback, adjust.' Ceremonies are optional; the feedback loop isn't.
- Measure the system (DORA), never rank the person (Goodhart) — and pair speed with stability.
- Next: what AI just changed — live, on screen — then the version of all of this that fits on an index card.

BLOCK 4

PART 4 OF 5 · THE AGENTIC SHIFT

From typing code to judging it.



● CHAT POLL · TAKE 60 SECONDS

Where has AI coding helped you most — and where has it burned you?

30 seconds with your neighbour: one win, one burn — then the room
hears a few of each.

| *Hold onto the burns — this section is about why they happen, and the guardrails real teams built in response.*

The work moved up the stack.

- **Claude Code, Cursor, Copilot, Codex** moved engineers from *typing* code to *specifying, steering and reviewing* it.
- Boilerplate, tests, glue, scaffolding — largely delegated. A day of scaffolding is now an hour.
- No longer fringe: **Google reports ~75% of its new code is AI-generated** (2025, human-reviewed) — at Anthropic it's over 80% (2025).

| *AI didn't replace the engineer — it moved the job to intent and judgment.*

But the data is messier than the hype.

SIGNAL	WHAT THE RESEARCH SAYS
Adoption	~90% of developers use AI at work (DORA 2025)
Perceived productivity	80%+ say AI makes them more productive (DORA 2025)
Trust in the output	Only ~24% highly trust AI-generated code — and trust is <i>falling</i> (DORA · Stack Overflow 2025)
Measured speed	Experienced devs <i>measured</i> 19% slower, <i>felt</i> 20% faster (METR 2025 — opening Story 2)
Team delivery	As AI adoption rose, throughput & stability initially <i>dropped</i> (DORA 2024)
Code quality	Copy-paste duplication up 8×; refactoring collapsed (GitClear 2024)

| *Individual speed ≠ team delivery. AI amplifies the system it lands in — a strong system gets stronger; a weak one gets faster at making a mess.*

The bottleneck moved to review — and teams got smaller.

- Teams now **generate code faster than they can safely review it** — the review queue is the new chokepoint. (Part 3 said the slow part is waiting; AI made the typing free and the waiting worse.)
- The floor of output rose, the **judgment gap widened** — the new failure mode is confident, plausible, *wrong* code nobody fully understands: **vibe-coding debt**.
- Same work, fewer people: 8–10-person teams trend toward 4–6 — the **n^2 problem run in reverse**. Small teams got superpowers.
- Hiring shifts toward judgment — people who can **spec, review, and integrate**. ⚠️ The industry's risk: cut juniors too deep and the pipeline of future seniors dries up.

| *Your edge entering the field now: AI fluency plus delivery judgment — scarcer than either alone. That combination is exactly this session.*

Our hardest blocker wasn't technical — it was fear.

- Some engineers were genuinely afraid AI would **take their jobs** — so, consciously or not, they pushed back against it.
- It showed up in **PR reviews**: AI-generated code got reviewed far more harshly than the same code from a human.
- **Nitpicking exploded** — naming, style, formatting — things that don't change whether the code *works*.
- The tell: even when the AI wrote *better* code, it got marked down for not matching our existing — often **poorer** — conventions.

| *When people feel threatened, they raise the bar selectively — and the AI's work quietly pays the tax.*

The fix: grade code on outcomes, not taste.

01 Functionality

Does it do the job, correctly — edge cases included?

02 Performance / SLA

Does it meet our latency & throughput targets?

03 Security

Is it safe to ship — vulnerabilities, data handling?

| *Code is increasingly written and maintained by AI — grading it on human style optimises for a reader that's disappearing. Hold the line on outcomes; let go of the rest.*

Tests are the trust mechanism.

- Outcome-based review needs an **objective referee** — not opinions. That's tests.
- **Deterministic tests are the one thing everyone can trust:** they pass or fail, identically, every time — no matter who or what wrote the code.
- One suite covers all three outcomes: **functionality** (does it work) · **performance** (assert the SLA) · **security** (tests + scans).
- AI flipped the economics: tests were the tedious work nobody did. **Now they're cheap** — near-total coverage is finally realistic.

| *Deterministic tests replace human nitpicking with objective truth — and AI made them cheap enough to actually have.*

Live demo: the same task, two specs.

SPEC A — VAGUE (WATCH IT GUESS)

"Write code to deduplicate our customer list." · Watch what the agent silently *assumes*: what counts as a duplicate? which record wins? where does the output go? · Confident output — unvalidated guesses. **The Expert, in reverse.**

SPEC B — CRISP (WATCH IT DELIVER)

Input: `customers.csv` — name, email, phone, `updated_at` · Duplicate = same email, OR same name + phone · Keep the newest record; log every merge · Must include tests · Must **not** modify the original file

Opening Story 1: one 'simple' feature, twelve repos.

LAYER	REPOS	CHANGE FOR 'ADD A DELIVERY-STATUS TYPE'
Proto / schema	2	New enum values, event definitions
Backend services	4	Business logic, handlers, DB migrations
Frontend apps	3	UI components, state
Shared libraries	2	Models, validation
Infra / docs	1	Pipeline config, documentation

12 repos, 12 PRs, 12 reviews, 12 deploys — in lockstep. Why 12 repos at all? Conway's Law from Part 1: the org's shape, printed onto the code. Right for human teams...

...but an agent in one repo sees a twelfth of the problem.

- AI coding is only as good as the **context it can see**. Give it the right context and it's brilliant; starve it and it *guesses* — confidently. (You just watched Spec A do exactly this.)
- Split this granularly, any single repo shows **only a fraction of the change**. The agent opens the backend repo and literally can't see the proto, the frontend, or the library it must change in step.
- It needs **context windows, not context switches**. Across twelve repos it sees none of the connections — so it can't reason about the feature as a whole.

| *The architecture that was legible to human teams is illegible to an agent.*

So we restructured the code for a new reader.

- Consolidated **tightly-coupled** services into **domain monorepos** — a whole change now lives in one place the agent can see at once.
- The agent reasons about the **entire feature end-to-end**, not 1/12 of it.
- **⚠ Not everything** — truly independent services stayed separate. The goal is legibility, not one giant repo.
- The trade-offs: heavier tooling, slower CI at scale, blurrier ownership. **Consolidate what changes together; separate what evolves independently.**

| *We used to structure code for human teams. Now we also structure it for agents. (That's opening Story 1, resolved.)*

What we rebuilt around the tools.

- **Specs first** — spec-driven development: define the outcome before the agent builds. (GitHub Spec Kit, OpenSpec, Amazon Kiro — this went mainstream in the past year.)
- **Context as infrastructure** — CLAUDE.md / AGENTS.md files, architecture docs the agents actually read. Documentation matters *more* now.
- **A bigger safety net** — near-total test coverage, stronger CI, security scanning and secret detection turned up.
- **Review re-scoped** — smaller PRs, AI-assisted review, a named human still accountable.

| *The tools are cheap; the guardrails are the real work. Everything from Parts 1–3 became more important, not less.*

BLOCK 5

PART 5 OF 5 · YOUR TOOLKIT

Agile for a team of one — plus agents.

Right-size the process — add it only when it hurts.

- **Minimum viable process for one person:** a list with three columns — Now / Doing / Done — and a 15-minute weekly review. That's complete and legitimate.
- Lost track of what's in flight? → **add a board**. Shipped the same bug twice? → **add a test or a checklist**.
- Forgetting why you decided something? → **a one-line decision log**. Working with one other person now? → **a weekly sync + a shared list**.

| *Process is a response to a specific pain — not a costume you wear to look professional.*

Tools you'll actually use.

TOOL	BEST FOR	WHY
Notion	Project + docs + DB in one	Run everything in one place
Linear	A board that feels pro	Free tier, fast, keyboard-driven
Trello	Dead-simple kanban	Lowest barrier — drag cards, done
GitHub Projects	Planning next to code	Free, lives with the repo
Todoist / Obsidian	Personal tasks / notes	Your own layer, separate from boards

⚠️ *Don't tool-hop. Pick one, use it for a month. The habit matters, not the tool. (And no — you don't need Jira.)*

Use AI as the team you don't have.

01 Plan

"Break this goal into a Now / Next / Later backlog."

02 Architect

"What are the pieces — and what could go wrong?"

03 Build

Generate the slice; you steer and review.

04 Review

"What did I miss — bugs, edge cases, security?"

| *Spec → plan → build → review — the same loop you watched in the demo. You're the only human in it, so keep the guardrails: never ship what you can't explain.*

Your weekly 15-minute review.

01 Look back

What shipped? What stalled, and why?

02 Two metrics

Cycle time trending down? Did anything break? (Your whole DORA dashboard.)

03 One improvement

Pick a single thing to change next week.

04 Re-stack

Drag Next → Now for the week ahead.

Fifteen minutes a week is the entire management overhead of a team of one — retro, planning and metrics review in one coffee.

• ✎ EXERCISE 3 • 10 MIN • SOLO, THEN PAIRS

Plan your project — right now.

Use a real project — your capstone, a portfolio piece, anything you actually want to ship. (No project yet? Plan the first week of your capstone, or use the stock project on the handout.)

- **1 • Backlog (4 min):** Now / Next / Later — two items each. A 'Now' item must be small enough to ship inside a week.
- **2 • Metrics (1 min):** write your two numbers — cycle time (idea → shipped) and did-my-last-change-break-anything.
- **3 • Spec one slice (3 min):** for your first 'Now' item, write three sentences an agent could build from — what it is, what 'done' looks like, what it must *not* do. (Spec B, not Spec A.)

| Then 2 min: swap with your neighbour. If they can tell what your first slice is, an agent can too.

Agile-for-one.

01 A tiny backlog

Now / Next / Later. (← roadmaps)

02 Small batches

Ship something every few days. (← Agile's core bet)

03 A retro with yourself

15 min weekly: what worked, what to change. The load-bearing habit.

04 Two metrics

Cycle time, and did-it-break. (← DORA, scaled to one)

05 AI as your team

Plan, build, review with agents — inside the guardrails. (← Part 4)

| *The 200-person playbook, scaled down to one — it fits on an index card.*

Today, in five lines.

- Coordination is the hard part — small teams, clean boundaries, Conway's Law.
- Agile is 'ship small, get feedback, adjust' — the loop, not the rituals.
- Measure the system, never the person — and pair speed with stability.
- AI moved the work from typing to judging — specs, tests and review are the new craft.
- Right-size everything — your version fits on an index card.

| *The goal isn't to run 200 engineers — it's to run the best team of one, and to recognise good engineering when you see it.*

TAKE IT FURTHER (FLEX SLIDE — IT'S IN THE SHARED DECK)

Five books, if you want to go deeper.

BOOK	WHY
Accelerate	The DORA research — the metrics that actually matter
Team Topologies	How to structure teams that ship
The Mythical Man-Month	Why adding people doesn't add speed
The Manager's Path	The IC → manager → leadership journey we skipped
Shape Up (free online)	A pragmatic alternative to fixed sprints

| *Start with whichever matches the problem in front of you.*

Questions?

Run the best team of one anyone's ever seen.